Jose Vargas
Fall 2026
09/27/2026
Linux Administration
Prof: Adrianna Holden-Gouveia

awk

AWK is a command-line text-processing tool that allows us to analyze, filter, and manipulate structured data by working with records and fields.

Exercise 1

Task

- Print all First Names.

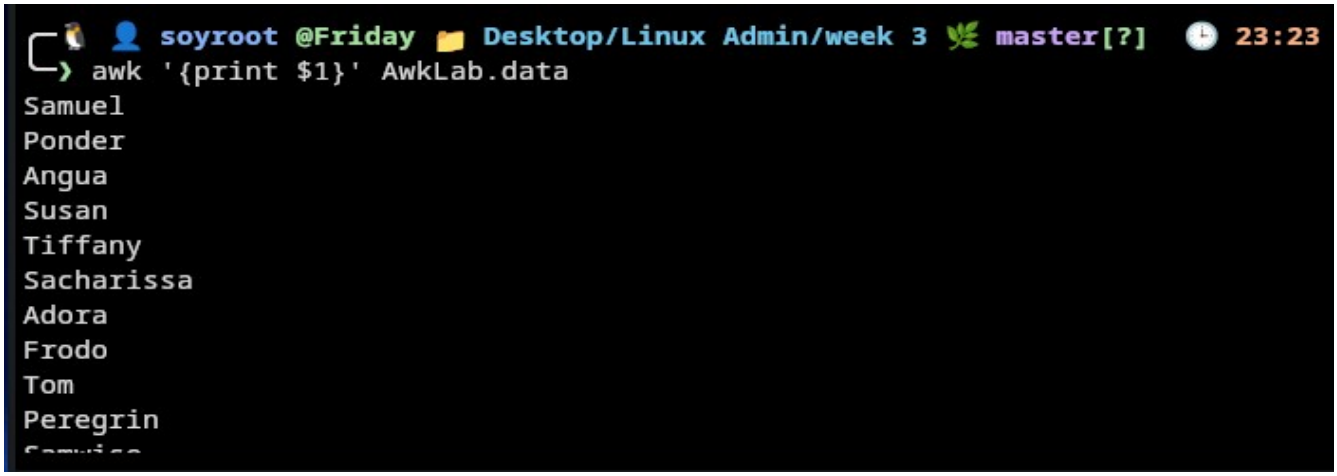
Command

- awk '{print $1}' AwkLab.data

How it works

- By default, AWK separates each record into fields and assigns each field a fixed variable, with $1 representing the first field. AWK also processes every record in the file by default. I used the basic AWK structure awk '{action}' file, using print $1 as the action to print the first field of each record.

Result



Exercise 2

Task

- Print phone numbers for Tom and Frodo after their names

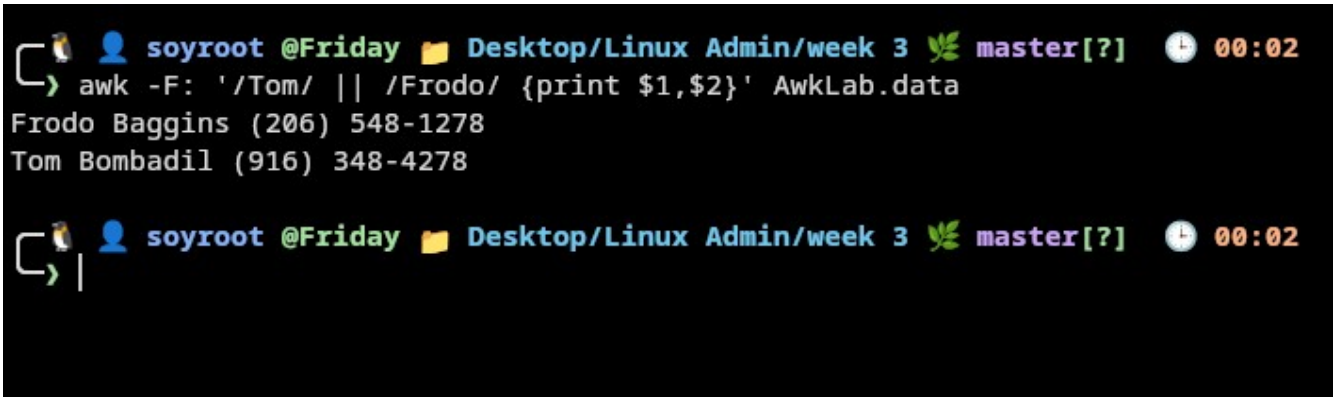
Command

- awk -F: '/Tom/ || /Frodo/ {print $1,$2}' AwkLab.data

How it works

- -F is used to change the field separator used to separate each field. In this case, : is used because of the structure of the data, allowing the name and phone number to be treated as separate fields. /Tom/ and /Frodo/ are patterns used to find records containing those values, and || is the OR operator, allowing either pattern to match. Finally, print $1,$2 prints the two fields separated by a space.

Result



Exercise 3

Task

- Print Peregrin’s full name and phone number area code only.

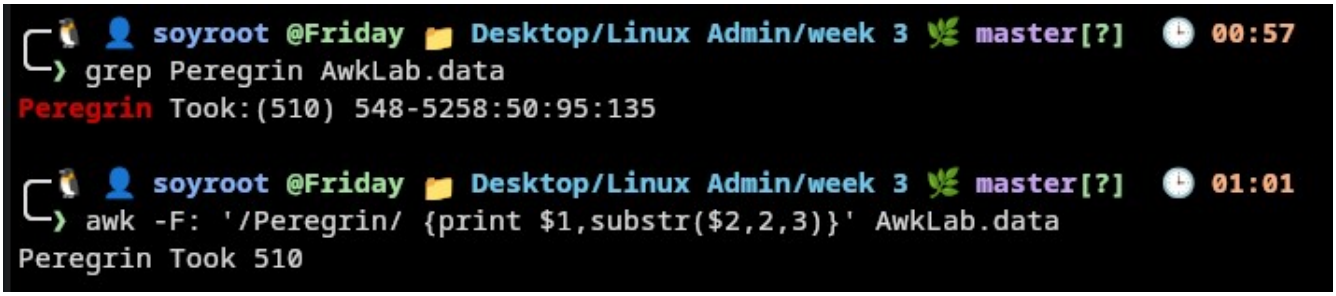
Command

- awk -F: '/Peregrin/ {print $1,substr($2,2,3)}' AwkLab.data

How it works

- Changing the field separator makes each full name field $1. The /Peregrin/ pattern is used to select the specific record containing Peregrin. With the phone number isolated in $2, substr($2,2,3) extracts three characters starting at the second character of the field, returning the area code without the parentheses.

Result



Exercise 4

Task

- Print all phone numbers (full number) in the 123 area code along with the names

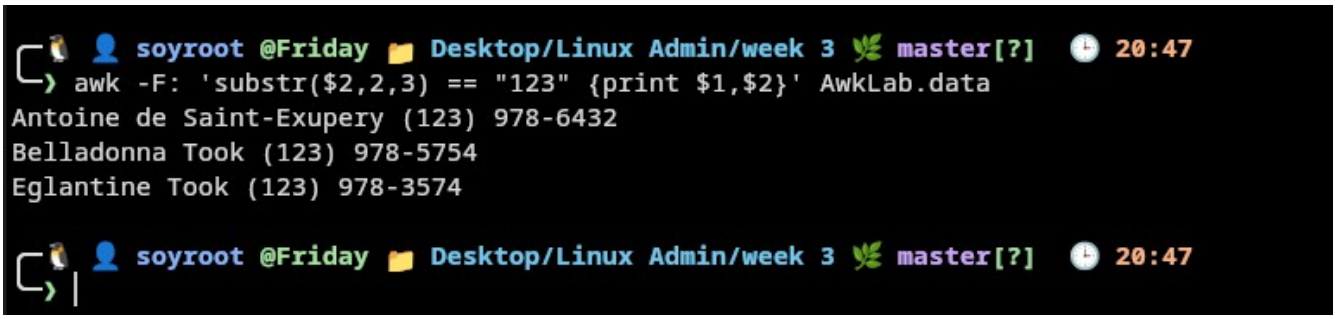
Command

- awk -F: 'substr($2,2,3) == "123" {print $1,$2}' AwkLab.data

How it works

- Using -F: separates the full name into $1 and the phone number into $2. substr($2,2,3) extracts three characters from the phone number starting at the second character. The condition == "123" compares the extracted area code with 123, so only records with that area code are processed. print $1,$2 then outputs the full name and phone number from the matching records.

Result



Exercise 5

Task

- 5. Print all Last names beginning with either a T or D (careful of middle names!)

Command

- awk -F: {'n = split($1, name, " "); if (substr(name[n],1,1) == "T" || substr(name[n],1,1) == "D") {print name[n]}}' AwkLab.data

How it works

- Using -F: makes the full name $1. The split() function takes three arguments: a string, an array, and a separator. n=split($1,name," ") splits the full name by spaces, stores each part in the name array, and stores the number of elements in n. Since n is also the index of the last element, name[n] represents the last name regardless of middle names. The if condition uses substr(name[n],1,1) to extract the first character of the last name and compares it with "T" or "D". If either condition is true, print name[n] outputs the matching last name.

Result



Exercise 6

Task

- 6. Print all first names containing four or less characters.

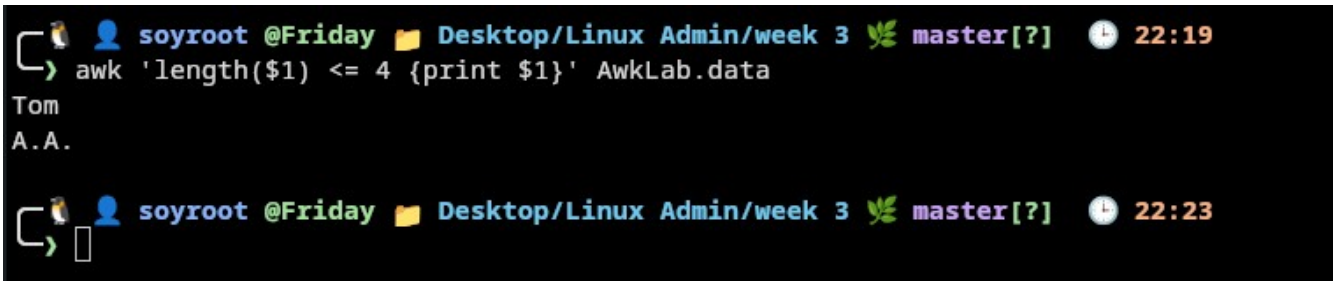
Command

- awk 'length($1) <= 4 {print $1}' AwkLab.data

How it works

- The length() function returns the number of characters in a string. In this case, $1 is passed to length() because the first field contains the first name. The condition length($1) <= 4 selects records where the first name contains four or fewer characters, and print $1 outputs the matching first names.

Result



Exercise 7

Task

- 7. Print the first names and area codes of all those in the 916 area code.

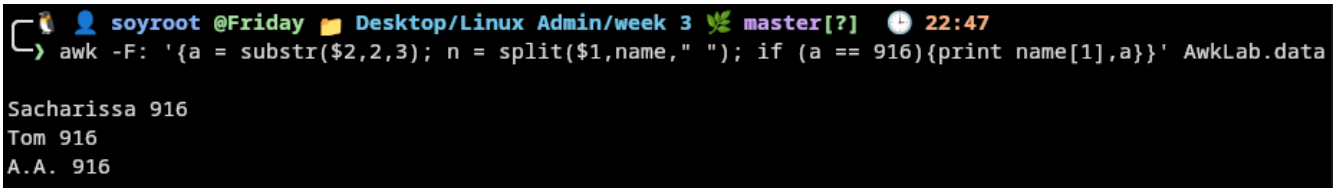
Command

- awk -F: '{a = substr($2,2,3); n = split($1,name," "); if (a == 916){print name[1],a}}' AwkLab.data

How it works

- -F: sets the field separator to :, making the full name $1 and the phone number $2. The variable a stores the result of substr($2,2,3), which extracts three characters starting at the second character of the phone number to obtain the area code. split($1,name," ") separates the full name by spaces and stores each part in the name array. The if condition compares a with 916. When the condition is true, print name[1],a outputs the first element of the name array, which is the first name, followed by the area code.

Result



Exercise 8

Task

- 8. Print Sacharissa’s campaign contributions following her name. Each value should be printed with a leading dollar sign; e.g., $250 $100 $175.

Command

- awk -F: '/Sacharissa/ {print $1,"$"$3,"$"$4,"$"$5}' AwkLab.data

How it works

- -F: sets the field separator to :. Based on the structure of the data, $1 contains the full name, while $3, $4, and $5 contain the campaign contributions. The /Sacharissa/ pattern selects her record, and print outputs her name followed by the three contribution fields. A dollar sign is placed inside double quotes ("$") so it is treated as a literal character and printed before each contribution.

Result



Exercise 9

Task

- 9. Print last names followed by a comma and the phone number. Be careful of the last names’s format.

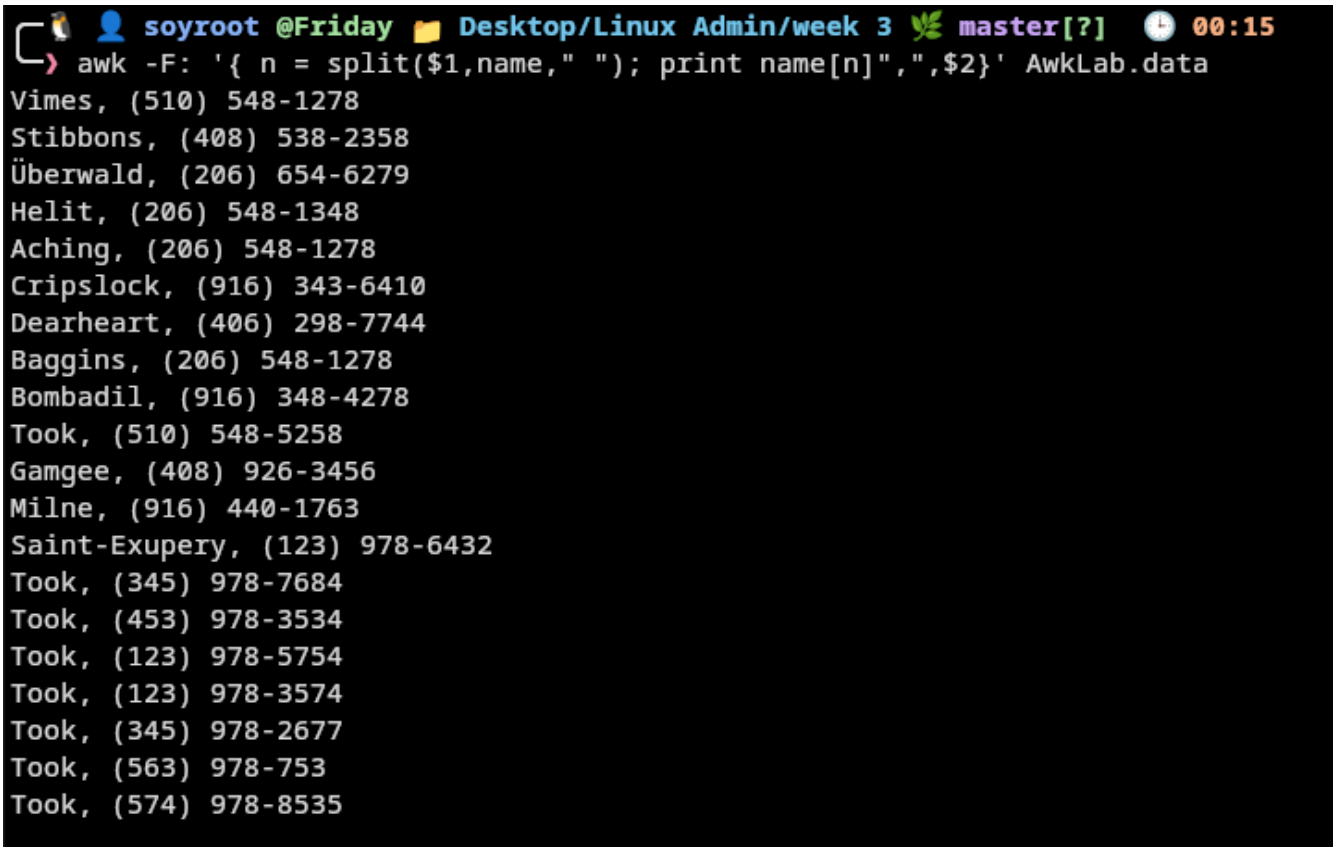
Command

- awk -F: '{ n = split($1,name," "); print name[n]",",$2}' AwkLab.data

How it works

- -F: sets the field separator to :, making the full name $1 and the phone number $2. split($1,name," ") separates the full name by spaces and stores each part in the name array. The number of elements returned by split() is stored in n, allowing name[n] to always reference the last element of the array regardless of middle names. print name[n]",",$2 then prints the last name followed by a literal comma and the phone number.

Result



Exercise 10

Task

- 10. Print the first and last names of those who contributed more than $110 in the last month. Make sure to include their last month contribution amount after the name

Command

- awk -F: '{n = split($1,name," "); if($5 > 110) print name[1],name[n],"$"$5}' AwkLab.data

How it works

- -F: sets the field separator to :, separating the name, phone number, and contributions based on the structure of the file. split($1,name," ") separates the full name by spaces and stores each part in the name array. The number of elements returned by split() is stored in n, making name[1] the first name and name[n] the last name. Based on the structure of the data, $5 contains the last-month contribution. The if ($5 > 110) condition checks whether this contribution is greater than $110. If the condition is true, the first name, last name, and last-month contribution are printed.

Result



Exercise 11

Task

- 11. Print the last names, phone numbers, and first month contribution of those who contributed less than $150 in the first month.

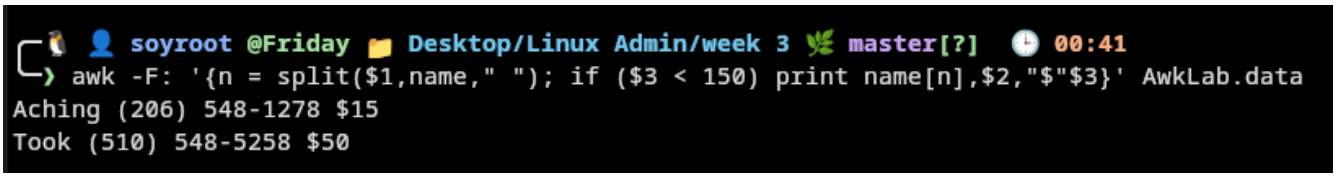
Command

- awk -F: '{n = split($1,name," "); if ($3 < 150) print name[n],$2,"$"$3}' AwkLab.data

How it works

- -F: sets the field separator to : based on the structure of the file. split($1,name," ") separates the full name into the name array, and the number of elements is stored in n, allowing name[n] to reference the last name. $3 contains the first-month contribution, so if ($3 < 150) checks whether it is less than $150. If the condition is true, the last name, phone number from $2, and first-month contribution from $3 are printed.

Result



Exercise 12

Task

- 12. Print the first names and contribution of those who contributed between $10 and $200 in the first month.

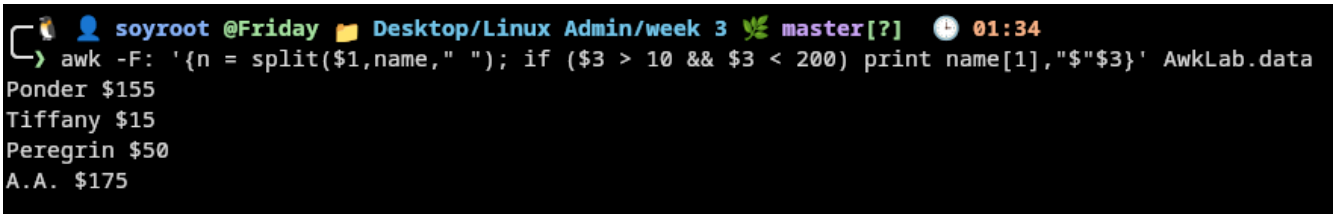
Command

- awk -F: '{n = split($1,name," "); if ($3 > 10 && $3 < 200) print name[1],"$"$3}' AwkLab.data

How it works

- -F: sets the field separator to : based on the structure of the file. split($1,name," ") separates the full name by spaces and stores each part in the name array. Since $3 contains the first-month contribution, the condition $3 > 10 && $3 < 200 checks that the contribution is greater than $10 and less than $200. Both conditions must be true because && represents the AND operator. If they are true, name[1] prints the first name followed by the first-month contribution from $3, with a literal dollar sign added before the amount.

Result



Exercise 13

Task

- 13. Print the first name, last names and total contributions of those who contributed less than $700 over the three-month period.

Command

- awk -F: '{n = split($1,name," "); t = $3 + $4 + $5; if (t < 700) print name[1],name[n],"$"t}' AwkLab.data

How it works

- -F: sets the field separator to : based on the structure of the file. split($1,name," ") separates the full name into the name array, allowing name[1] to reference the first name and name[n] the last name. The variable t stores the total contribution by adding the three contribution fields, $3 + $4 + $5. The condition if (t < 700) checks whether the total contribution over the three-month period is less than $700. If true, the first name, last name, and total contribution are printed.

Result

```
soyroot @Friday Desktop/Linux Admin/week 3 master[?] 01:42
❯ awk -F: '{n = split($1,name," "); t = $3 + $4 + $5; if (t < 700 ) print name[1],name[n],"$"t}' AwkLab.data
Samuel Vimes $525
Ponder Stibbons $446
Angua Überwald $360
Susan Helit $525
Tiffany Aching $353
Sacharissa Cripslock $525
Frodo Baggins $405
Tom Bombadil $525
Peregrin Took $280
Samwise Gamgee $618
A.A. Milne $550
Antoine Saint-Exupery $525
```

Exercise 14

Task

- 14. Print the first names and first letter of the last name, and average contribution of those who had an average contribution of more then $300

Command

- awk -F: '{n = split($1,name," "); l = substr(name[n],1,1); avg = ($3 + $4 + $5) / 3; if (avg > 300) print name[1],l,"$"avg}' AwkLab.data

How it works

- -F: sets the field separator to : based on the structure of the file. split($1,name," ") separates the full name into the name array, where name[1] represents the first name and name[n] the last name. substr(name[n],1,1) extracts the first character of the last name and stores it in l. The variable avg calculates the average contribution by adding the three contribution fields, $3, $4, and $5, and dividing the total by 3. The condition if (avg > 300) selects only records with an average contribution greater than $300. If true, the first name, first letter of the last name, and average contribution are printed.

Result

```
soyroot @Friday Desktop/Linux Admin/week 3 master[?] 02:14
❯ awk -F: '{n = split($1,name," "); l = substr(name[n],1,1); avg = ($3 + $4 + $5) / 3; if (avg > 300 ) print name[1],l,"$"avg}'
 AwkLab.data
Adora D $341.667
Adalgrim T $1746.67
Bandobras T $3931.33
Eglantine T $1771.67
Mirabella T $507
Ferumbras T $1269
Gerontius T $1925
```

Exercise 15

Task

- 15. Print the last name and area code of those not in the 916 area code.

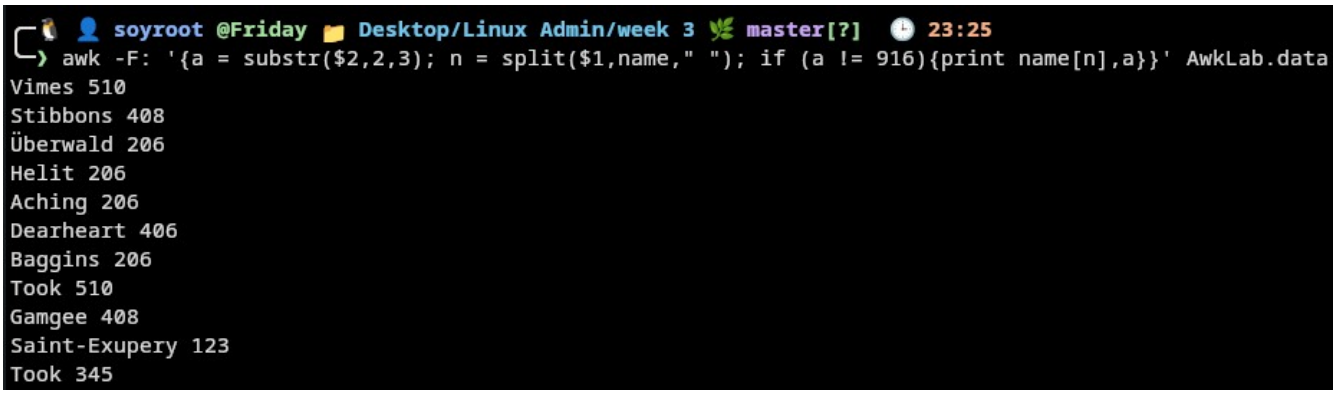
Command

- awk -F: '{a = substr($2,2,3); n = split($1,name," "); if (a != 916){print name[n],a}}' AwkLab.data

How it works

- -F: sets the field separator to :. substr($2,2,3) extracts the area code from the phone number and stores it in a. split($1,name," ") separates the full name into the name array, allowing name[n] to reference the last name. The condition if (a != 916) uses the not-equal operator (!=) to select records whose area code is not 916. The last name and area code are then printed.

Result



Exercise 16

Task

- 16. Print each record preceded by the number of the record.

Command

- awk '{print NR, $0}' AwkLab.data

How it works

- NR is a built-in AWK variable that contains the number of the current record being processed. $0 represents the entire current record. By printing NR followed by $0, each record is displayed with its corresponding record number before it.

Result

```
soyroot @Friday Desktop/Linux Admin/week 3 master[?] 02:19
❯ awk '{print NR, $0}' AwkLab.data
1 Samuel Vimes:(510) 548-1278:250:100:175
2 Ponder Stibbons:(408) 538-2358:155:90:201
3 Angua von Überwald:(206) 654-6279:250:60:50
4 Susan Sto Helit:(206) 548-1348:250:100:175
5 Tiffany Aching:(206) 548-1278:15:188:150
6 Sacharissa Cripslock:(916) 343-6410:250:100:175
7 Adora Belle Dearheart:(406) 298-7744:450:300:275
8 Frodo Baggins:(206) 548-1278:250:80:75
9 Tom Bombadil:(916) 348-4278:250:100:175
10 Peregrin Took:(510) 548-5258:50:95:135
11 Samwise Gamgee:(408) 926-3456:250:168:200
12 A.A. Milne:(916) 440-1763:175:75:300
13 Antoine de Saint-Exupery:(123) 978-6432:250:100:175
14 Adalgrim Took:(345) 978-7684:4673:100:467
15 Bandobras "Bullroarer" Took:(453) 978-3534:6753:368:4673
16 Belladonna Took:(123) 978-5754:356:247:175
17 Eglantine Took:(123) 978-3574:473:475:4367
18 Mirabella Took:(345) 978-2677:783:563:175
19 Ferumbras III Took:(563) 978-753:250:100:3457
20 Gerontius Took:(574) 978-8535:535:678:4562
```

Exercise 17

Task

- 17. Print the name and total contribution of each person.

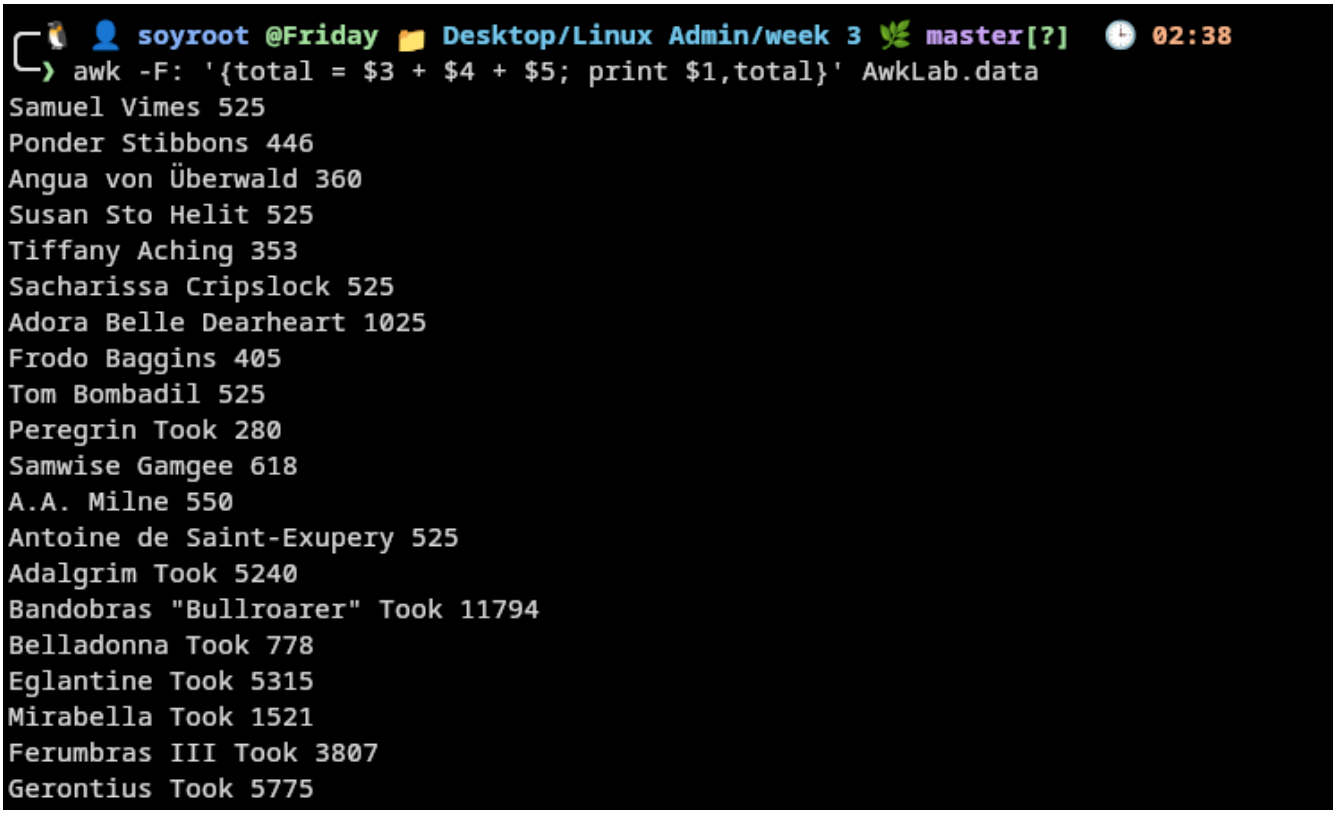
Command

- awk -F: '{total = $3 + $4 + $5; print $1,total}' AwkLab.data

How it works

- -F: sets the field separator to : based on the structure of the file, making the full name $1 and the three contribution amounts $3, $4, and $5. The variable total stores the sum of the three contribution fields. print $1,total then prints each person's full name followed by their total contribution.

Result



Exercise 18

Task

- 18. Add $10 to Tiffany Aching's first contribution and print her full name and first contribution.

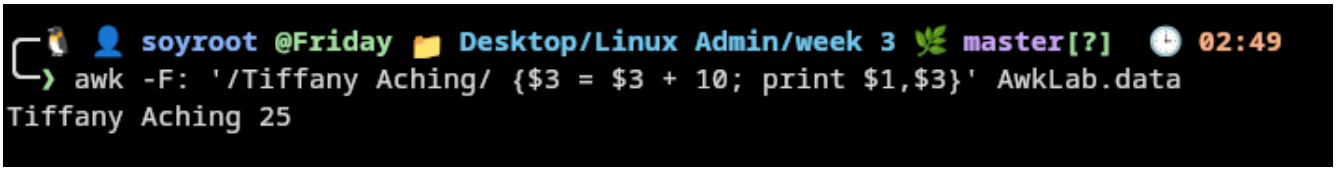
Command

- awk -F: '/Tiffany Aching/ {$3 = $3 + 10; print $1,$3}' AwkLab.data

How it works

- -F: sets the field separator to : based on the structure of the file. The /Tiffany Aching/ pattern selects only Tiffany Aching's record. $3 contains the first contribution, and $3 = $3 + 10 adds $10 to its current value and stores the new value back in $3. print $1,$3 then prints her full name and the updated first contribution. This changes the field while AWK processes the record but does not modify the original file.

Result



Exercise 19

Task

- 19. Change Samwise Gamgee's name to Sean Astin

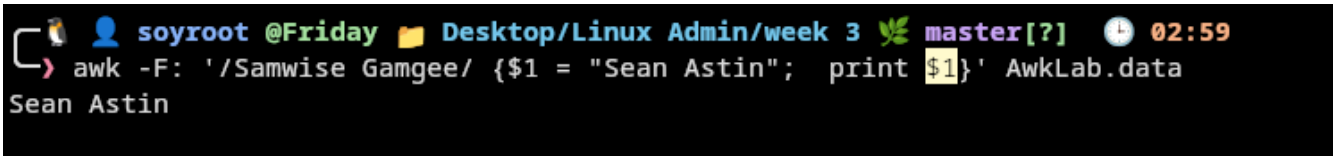
Command

- awk -F: '/Samwise Gamgee/ {$1 = "Sean Astin"; print $1}' AwkLab.data

How it works

- -F: sets the field separator to : based on the structure of the file. The /Samwise Gamgee/ pattern selects the record containing his name. $1 contains the full name, so $1 = "Sean Astin" replaces its value with the new name. print $1 then prints the modified field to show the result.

Result



Exercise 20

Task

- 20. Write an awk script to do the following (MUST be an awk script not just a bash script or commands on the commandline)

  (a) Prints first name of the all the Tooks followed by their total campaign contributions .

  (b) Print the full names and contributions of anyone who contributed between $10 and $200 in the last contribution

  (c) Prints the full names and average contribution of those who contributed less than $300 on average

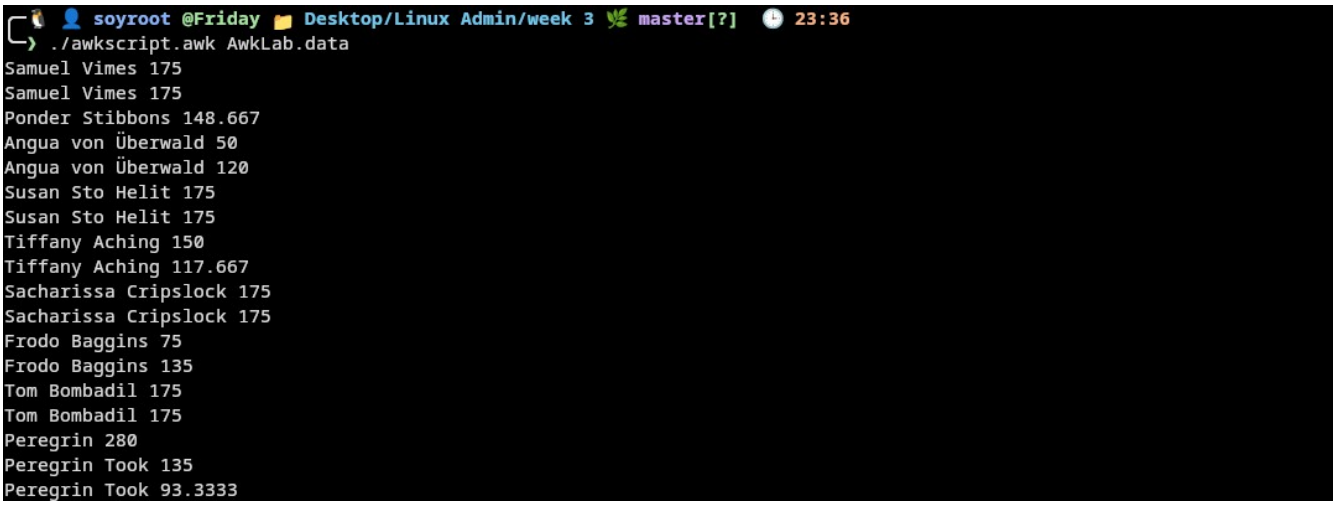
Command

- ./awkscript.awk AwkLab.data

How it works

- This exercise uses an AWK script instead of running the program directly from the command line. The shebang specifies /usr/bin/awk -f as the interpreter, allowing the script to be executed directly. The BEGIN block sets : as the field separator before processing the data. Part A uses split() to identify the first and last names and calculates the total contribution. Part B checks the last contribution using $5 and the && operator. Part C calculates the average of the three contributions and checks whether it is less than $300.

Result



Sources

- Grymoire. “AWK.” Unix/Linux Awk Tutorial. https://www.grymoire.com/Unix/Awk.html
- Holden-Gouveia, Adrianna. “Awk.” Linux Administration. https://www.aholdengouveia.name/LinuxAdmin/Awk.html#top